



PROJECT-BASED LEARNING

PHASE-I EVALUATION

**EcoGraph – Graph Based
Algorithmic Waste Management**

Team ID: DSCPP-III-2026-T186

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T186
Semester: 3rd
Section: A / D / H
Domain: Graph Algorithms (C++)

Team Members

1. Tejas Jain – 2028228 (SEC-A)
2. Yash Nigam – 2028289 (SEC-H)
3. Apoorv Negi – 2027640 (SEC-D)

Mentor

Navneet Rajput

Interactions completed: _____

Approved Project Name: EcoGraph – Graph Based Algorithmic Waste Management.

Problem Statement & Motivation

Problem Statement

Municipal waste routes are typically **static and fixed**, ignoring the real shortest route between homes, bins, and the depot – wasting fuel, time, and labor. Hazardous waste is handled identically to ordinary waste despite needing immediate, restricted-zone pickup, and fixed schedules over- or under-service locations with different fill rates.

Motivation

- ▶ Fixed routes ignore the real shortest path in the city's road/graph network
- ▶ Hazardous waste in normal queues risks delayed, unsafe transit
- ▶ Calendar-based schedules waste labor on locations nowhere near full

Target Users / Stakeholders

- ▶ Municipal waste collection departments and truck operators
- ▶ Residents and commercial establishments being served
- ▶ Public health – faster, safer hazardous waste clearance

Objectives

Project Objectives

1. Model the city as a weighted graph – depot, homes, junctions, and bins – with every node and edge tagged by zone type (residential, commercial, industrial).
2. Compute optimal collection routes using Dijkstra's shortest-path algorithm, with two selectable strategies: Priority mode and FIFO mode.
3. Detect hazardous waste and dispatch it immediately via a zone-constrained shortest path that avoids residential and commercial areas.
4. Dynamically determine collection frequency per location from its fill-rate, instead of a fixed calendar.
5. Expose all functionality through a REST API and visualize live, animated route computation on a web frontend.

Proposed Solution – Approach

Proposed Approach

1. **City layout** – depot, homes, junctions, bins, and zone tags loaded from a JSON configuration file at server startup.
2. **Routing engine** – Dijkstra's algorithm (plain and zone-constrained variants) computes shortest paths between any two nodes.
3. **Route builders** – Priority and FIFO strategies sequence multi-stop collection routes.
4. **Scheduler** – fill-rate and last-collected timestamps determine which locations are due for a visit.
5. **Delivery** – results served as JSON over a REST API, animated live on a Cytoscape.js graph canvas.

A single persistent C++ HTTP server owns the graph and performs routing, classification, and scheduling.

Proposed Solution – Key Features

Key Features

- ▶ Graph-based city model with zone tagging
- ▶ Shortest-path routing (Priority / FIFO toggle)
- ▶ Rule-based waste classification (keyword matching)
- ▶ Hazardous waste: immediate, zone-constrained routing bypassing the normal queue
- ▶ Dynamic collection frequency driven by per-location fill-rate
- ▶ Live route animation with hazardous routes visually distinguished

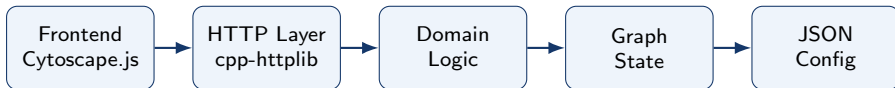
Every feature maps directly to one of the five REST endpoints and one domain-layer C++ class – see slides 10–14.

Integration of PBL Course Concepts

Course	Concepts Applied
Data Structures in C	Graph (adjacency list), Queue (FIFO), Priority queue / min-heap, Hash map (<code>unordered_map</code>), Path reconstruction (<code>vector/stack</code>)
OOPs with C++	Classes for Graph, Node, Edge, Scheduler, Router; encapsulation; STL containers throughout the domain layer

Integration: The city graph (DSA: adjacency list, hashing, heaps, queues) is implemented as encapsulated C++ classes (OOP) exposing Dijkstra, routing, and scheduling as one cohesive backend.

System Architecture Overview



- ▶ Single persistent process owns the graph in memory – fast, no per-request re-parsing
- ▶ Layered backend: HTTP layer → controllers → pure, testable domain classes
- ▶ Same-origin static file serving + API – no CORS complexity

Architecture – Components & Data Flow

Major Components

- ▶ Graph – Node, Edge, zone tags, `unordered_map` adjacency list
- ▶ Routing engine – plain + constrained Dijkstra, Priority/FIFO route builders
- ▶ Classifier – keyword-rule waste categorization
- ▶ Scheduler – fill-rate driven due-date computation
- ▶ REST controllers – one handler per endpoint

Data / Control Flow

- ▶ Frontend calls `/route`, `/classify`, `/schedule`, `/route/hazard`
- ▶ Backend loads `city.json` once at startup
- ▶ Fill-rate / last-collected state mutates in memory as locations are visited
- ▶ Response: JSON path + cost, animated on the graph canvas

Domain Model – Core Data Structures

Node

- ▶ id, type (Depot/Home/Junction/Bin)
- ▶ zone (Residential/Commercial/Industrial/Arterial)
- ▶ fillRate, capacity, lastCollected

Edge

- ▶ to, weight (travel cost)
- ▶ zone tag of the road segment itself

Graph class

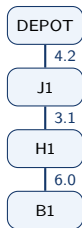
- ▶ Backed by `unordered_map<string, vector<Edge>>`
- ▶ $O(1)$ average node lookup, $O(\text{degree})$ edge iteration
- ▶ `loadFromJson()`, `neighbors()`, `toJson()`

```
{"id":"H1","type":"home","zone":"residential","fillRate":5.0,"capacity":50}
```

Structure / Algorithm	Short Definition	Used For
Graph (Adjacency List)	Each node holds a list of its outgoing weighted edges	City map: depot, homes, junctions, bins
Hash Map (unordered_map)	Key $\rightarrow$ value store, $O(1)$ average lookup	Node ID $\rightarrow$ adjacency list / node data
Queue (FIFO)	First element enqueued is the first dequeued	Sequences homes in FIFO collection mode
Priority Queue (Min-Heap)	Smallest-key element always dequeued next	Dijkstra's frontier – nearest node first
Priority Queue (Max-Heap)	Largest-key element always dequeued next	Scheduler – ranks locations by overdue-ness
Dijkstra's Algorithm	Greedy shortest path via min-heap relaxation	Depot / home / bin routing
Constrained Dijkstra	Dijkstra skipping forbidden-zone edges	Hazardous waste routing
Vector (Path Reconstruction)	Backtrack predecessor pointers into an ordered list	Rebuilds the final route

Graph Storage – Adjacency List Architecture

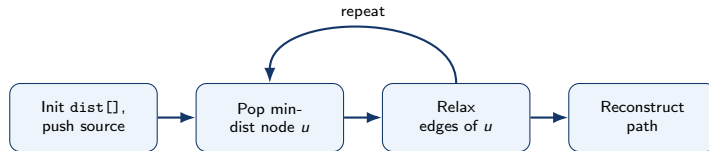
Backing store: `unordered_map<string, vector<Edge>>` $\rightarrow O(1)$ node lookup, $O(\text{degree})$ edge iteration.



Node	Adjacency List
DEPOT	(J1, 4.2, arterial)
J1	(DEPOT,4.2,art.), (H1,3.1,resid.)
H1	(J1,3.1,resid.), (B1,6.0,indust.)
B1	(H1, 6.0, industrial)

- ▶ Each Edge carries its own zone tag – lets the constrained variant treat it as impassable
- ▶ Loaded once from `city.json` at startup; edges default to bidirectional

Dijkstra's Algorithm – How It Works

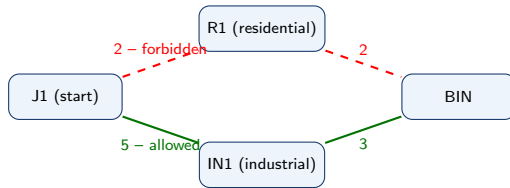


1. $\text{dist}[\text{source}] = 0$, all others ∞ ; push source into a min-heap keyed by distance.
2. Pop node u with smallest tentative distance.
3. **Relax** edge (u, v, w) : if $\text{dist}[u] + w < \text{dist}[v]$, update $\text{dist}[v]$ and push v .
4. Reconstruct the path by walking a predecessor map back to the source.

Complexity: $O((V + E) \log V)$ with a binary heap.

Custom Constrained Dijkstra for Hazardous Routing

EcoGraph adds one parameter, `forbiddenZones`: during relaxation, any edge whose zone is in this set is skipped entirely, as if it did not exist.

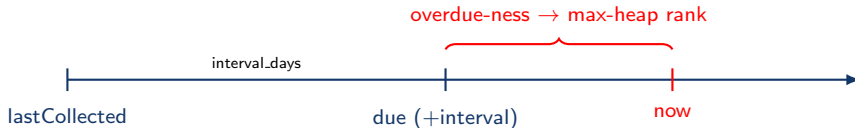


- ▶ Shorter path via R1 (cost 4) is residential – **skipped**; algorithm instead takes J1→IN1→BIN (cost 8) through industrial zones only
- ▶ Called with `forbiddenZones = {Residential, Commercial}` for every hazardous dispatch; returns HTTP 422 if no path exists

Fill-Rate & Frequency Management – Interval Formula

Each home/bin stores `fillRate` (units/day), `capacity`, and `lastCollected`. The scheduler turns these into a per-location visit interval:

```
interval_days = capacity / fillRate, clamped to [1,14] days
```



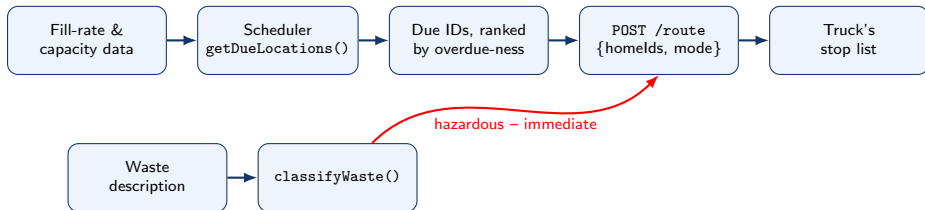
- ▶ **Floor of 1 day** – stops an extremely fast-filling bin from demanding more than one visit per day
- ▶ **Ceiling of 14 days** – guarantees no location goes unvisited more than two weeks, however slowly it fills
- ▶ Due when $\text{now} \geq \text{lastCollected} + \text{interval_days}$; `markCollected()` resets `lastCollected` = `now` once visited

Fill-Rate & Frequency Management – Worked Example

Location	Capacity	fillRate	interval_days	Last Collected	Status Today
H1	50	5/day	$50/5 = 10$	9 days ago	Not due (1 day left)
H2	100	20/day	$100/20 = 5$	6 days ago	Due – 1 day overdue
B1	300	40/day	$300/40 = 7.5$	10 days ago	Due – 2.5 days overdue
B2	100	2/day	$100/2 = 50$ → 14	20 days ago	Due – 6 days overdue

Max-heap serving order (most overdue first): **B2 (6d)** → **B1 (2.5d)** → **H2 (1d)**; H1 stays off today's list.

Selecting Truck Stops – Frequency + Waste Type



- ▶ Each run, the scheduler's due list (frequency-driven) becomes the `homeIds` array sent to POST /route – this is how truck stops are chosen
- ▶ If a description classifies as **hazardous**, its location skips the due list entirely and is dispatched immediately via POST /route/hazard
- ▶ Non-hazardous waste never overrides the schedule – its location simply waits for its fill-rate interval to elapse (previous slide)

Waste Classification Subsystem

- ▶ Keyword rules loaded from `waste_rules.json`: each category maps to a keyword list and a hazard flag.
- ▶ Categories: recyclable, compost, general, **medical**, **chemical**.
- ▶ A description is matched against **medical** and **chemical** keywords *first* – hazard classification always takes priority, even if a recyclable keyword also appears (e.g. “plastic syringe” → medical).
- ▶ Falls back to `general` if no keyword matches.
- ▶ Output: bin type + a boolean hazardous flag, which triggers the constrained-routing path instead of the normal queue/schedule.

REST API – How It Works & Design

The frontend and backend communicate over plain HTTP: the browser sends a JSON request to a resource path, the C++ server responds with a JSON body – no session state is kept on either side between calls.

Endpoint	Purpose
GET /graph	Full node/edge list as JSON, including zone tags
POST /route	Home list + mode (priority/fifo) → path & cost
POST /classify	Waste description → bin type & hazard flag
POST /route/hazard	Hazardous location → zone-constrained route
GET /schedule	Homes/bins due for collection today

Uniform error format {ok:false, error, message} with HTTP 400/404/422/500 as appropriate.

Frontend Visualization – Cytoscape.js

- ▶ **No React** – plain HTML/CSS/JavaScript served directly by the C++ server, so the whole project ships as one binary with no build step or bundler.
- ▶ **Cytoscape.js** – a graph-drawing library purpose-built for nodes-and-edges visualization, a closer fit here than a general-purpose UI framework.
- ▶ **Render pipeline:** GET /graph → build Cytoscape elements → style by node type/zone → auto-layout.
- ▶ **Route animation:** walk each returned path segment, sequentially highlighting edges and pulsing nodes; hazardous routes render in a distinct dashed/red style.
- ▶ **Live schedule:** periodic GET /schedule polling flags due locations on the map without a page reload.

Technology Stack

Backend

- ▶ Programming Language: C++
- ▶ HTTP Framework: `cpp-httplib` (single-header)
- ▶ JSON Library: `nlohmann/json` (header-only)
- ▶ Build System: CMake

Frontend & Tooling

- ▶ HTML / CSS / vanilla JavaScript
- ▶ Cytoscape.js for graph rendering and route animation
- ▶ Version Control: Git / GitHub

Development Methodology

1. **Foundations** – graph data model, JSON loader, plain Dijkstra, unit tests
2. **Core Logic** – constrained Dijkstra, classifier, Priority/FIFO routers, scheduler
3. **HTTP API** – all 5 REST endpoints wired and verified against sample data
4. **Frontend Integration** – Cytoscape.js rendering, animation, hazard styling, schedule view
5. **Stretch Goals** – thread-safety (`shared_mutex`), JSON-file state persistence

Each phase has an explicit exit criterion (e.g. “test binary prints correct shortest paths”) before the team moves to the next layer.

Initial Progress Achieved

- ▶ Background / literature study
- ▶ Problem statement and requirements identified
- ▶ System architecture prepared
- ▶ Technology stack finalized
- ▶ Implementation plan and team roles outlined

Individual Contribution

Member	Role	Contribution
Tejas Jain	Team Lead	Problem analysis, system architecture, REST API & domain model design, technology stack selection, project documentation and presentation
Apoorv Negi	Member	Supporting research on routing algorithms
Yash Nigam	Member	Supporting research on data requirements

Roadmap & Expected Outcomes

Roadmap

1. Phase-I: Proposal, Architecture & API Design
2. Phase-II: Graph Engine, Algorithms, REST API, Frontend
3. Phase-III: Full Live Demo, Stretch Goals

Expected Outcomes

- ▶ Working C++ backend with an animated web frontend
- ▶ Verified shortest-path and constrained-route correctness
- ▶ Demonstrated fill-rate-driven dynamic scheduling
- ▶ Extensible to concurrency and persistent storage

References

1. Cormen, Leiserson, Rivest, Stein – *Introduction to Algorithms* (Dijkstra's Algorithm)
2. cpp-http-lib – github.com/yhirose/cpp-http-lib
3. nlohmann/json – github.com/nlohmann/json
4. Cytoscape.js Documentation – js.cytoscape.org

Thank You
Questions & Suggestions